

Reviewer Pack — Bonami-Beckner 4-2 Saturation of Clause-Sum Bounds DPLL Dept...

Entry 2b2653395609 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 03:29:05 UTC

Bonami-Beckner 4-2 Saturation of Clause-Sum Bounds DPLL Depth

Entry ID: 2b2653395609 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 03:29:05 UTC

1. Conjecture

Field A (mathematical branch): Bonami-Beckner hypercontractivity (Bonami 1970, Beckner 1975) — the degree- d 4-norm-vs-2-norm contraction $\|P\|_4 \leq (q-1)^{\{d/2\}} \|P\|_2$ — applied as a SYNTACTIC Fourier-analytic invariant of the centered clause-indicator polynomial Q_F of a CNF (a property of the formula's clause/polarity structure, NOT of its truth table). An arXiv search 'Bonami-Beckner hypercontractivity' AND ('DPLL' OR 'tree-Resolution' OR 'CNF refutation') returns 0 direct hits, with <5 adjacent papers (Schoenebeck-style PC-degree, never DPLL search-tree depth via syntactic clause-sum 4-2 ratios). Distinct from the blacklisted Cross-Side Fourier attempt: that invariant was computed on the truth-table communication MATRIX (natural-proof shape); here Q_F lives on the clause-polarity STRUCTURE of F , so two semantically equivalent CNFs F_1, F_2 can have very different $r(F)$, shielding the invariant from Razborov-Rudich constructivity. **Field B** (complexity object): DPLL / tree-like Resolution refutation depth $d(F)$ for unsatisfiable 3-CNFs, in the Ben-Sasson-Wigderson-Krajíček bounded-arithmetic regime where $\log_2 d(F)$ controls V^0 -witnessing complexity of $\neg F$ as Σ^b_0 and lower-bounds tree-Resolution width $w^*(F)$.

Statement:

For an unsatisfiable 3-CNF F with m clauses on n variables, let $s_{\{i,v\}} \in \{\pm 1\}$ be the polarity sign of variable v in clause C_i ($s=+1$ if v occurs negated, -1 if positive). Define the centered clause-falsification polynomial $Q_F: \{-1, +1\}^n \rightarrow \mathbb{R}$ by $Q_F(x) = \sum_{i=1}^m \prod_{v \in \text{vars}(C_i)} (1 + s_{\{i,v\}} x_v) / 2 - 1/8$, whose Fourier coefficient at $T \subseteq [n]$ with $|T| \in \{1, 2, 3\}$ is the closed-form sum $\hat{Q}_F(T) = (1/8) \sum_{i: T \subseteq \text{vars}(C_i)} \prod_{v \in T} s_{\{i,v\}}$ (computable directly from clause structure in $O(m)$ time). Let $r(F) := \|Q_F\|_4 / \|Q_F\|_2 \in [1, 3^{\{3/2\}}]$ be the Bonami-Beckner 4-2 ratio, and set $G(F) := (3^{\{3/2\}} - r(F)) / 3^{\{3/2\}} \in [0, 1 - 3^{\{-3/2\}}]$. Conjecture: there is an absolute constant $c \geq 0.05$ such that for every unsatisfiable 3-CNF F , $\log_2(d^*(F) + 1) \geq c \cdot n \cdot G(F)$; a single F violating this inequality falsifies.

Rationale (proposer's reasoning):

Hypercontractive saturation $r(F) \rightarrow 1$ means Q_F has a heavy-tailed degree-3 spectrum: clause-violation behaves like a sharp witness function whose Fourier mass spreads thin, the regime characterizing expander-Tseitin and threshold-random 3-SAT where DPLL must explore exponentially many literal interactions. Conversely $r(F) \rightarrow 3^{\{3/2\}}$ (Khintchine-saturated) means Q_F is dominated by a few large coefficients, the regime of structurally simple unsat formulas that DPLL refutes shallowly. The bridge is novel

because the Fourier analysis is performed on the CLAUSE STRUCTURE polynomial Q_F (syntactic), not on F 's truth table or its communication matrix, sidestepping the natural-proofs largeness barrier that killed the Cross-Side Fourier attempt.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 91403c61299ba527

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $30 \text{ seeds} \times n \in \{10, 12, 14, 16\} \times 4 \text{ ensembles}$ (480 unsat 3-CNF instances), compute $G(F)$ via exact Parseval $\|Q_F\|_2^2$ and 5000-sample Monte Carlo $\|Q_F\|_4^4$, and measure $d(F)$ by DPLL with unit propagation. Supported iff every instance satisfies $\log_2(d(F)+1) \geq 0.05 \cdot n \cdot G(F)$; a single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Bonami-Beckner hypercontractivity DPLL tree-Resolution refutation depth-Fourier analysis CNF clause polynomial Resolution width lower bound-hypercontractive 4-2 norm ratio 3-CNF tree-like Resolution depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_3sat(n, m):
    F = []
    for _ in range(m):
        clause = []
        vars_in_clause = random.sample(range(1, n + 1), 3)
```

```

        for v in vars_in_clause:
            if random.random() < 0.5:
                clause.append(-v)
            else:
                clause.append(v)
        F.append(clause)
    return F

def generate_tseitin(n):
    F = []
    edges = list(itertools.combinations(range(1, n + 1), 2))
    for u, v in edges:
        x = len(F) + 1
        F.append([u, v, x])
        F.append([-u, -v, x])
        F.append([u, -v, -x])
        F.append([-u, v, -x])
    return F

def generate_ordering_principle(n):
    F = []
    for i in range(1, n):
        F.append([i, i + 1])
        F.append([-i, -(i + 1)])
    return F

def generate_pebbling(n):
    F = []
    for i in range(1, n):
        F.append([i, i + 1])
        F.append([-i, -(i + 1)])
    return F

def compute_q_hat(F):
    q_hat = defaultdict(float)
    for clause in F:
        vars_in_clause = [abs(lit) for lit in clause]
        signs = [1 if lit < 0 else -1 for lit in clause]
        for T in itertools.combinations(vars_in_clause, 1):
            q_hat[T] += signs[vars_in_clause.index(T[0])] / 8
        for T in itertools.combinations(vars_in_clause, 2):
            q_hat[T] += signs[vars_in_clause.index(T[0])] * signs[vars_in_clause.index(T[1])] / 8
        for T in itertools.combinations(vars_in_clause, 3):
            q_hat[T] += signs[vars_in_clause.index(T[0])] * signs[vars_in_clause.index(T[1])] * signs[vars_in_clause.index(T[2])] / 8
    return q_hat

def compute_norm_2(q_hat):
    norm_2_sq = sum(val ** 2 for val in q_hat.values())
    return math.sqrt(norm_2_sq)

def compute_norm_4(F, n, samples=5000):
    total = 0.0
    for _ in range(samples):
        x = [random.choice([-1, 1]) for _ in range(n)]
        q_val = 0.0
        for clause in F:

```

```

        product = 1.0
        for lit in clause:
            v = abs(lit)
            s = 1 if lit < 0 else -1
            product *= (1 + s * x[v - 1]) / 2
        q_val += product - 1/8
        total += q_val ** 4
    return (total / samples) ** (1/4)

def dpll_depth(F):
    def backtrack(assignment, clauses):
        if not clauses:
            return 0
        for clause in clauses:
            if all(lit in assignment for lit in clause):
                continue
            if any(-lit in assignment for lit in clause):
                continue
            break
        else:
            return 0

        unassigned_vars = set()
        for clause in clauses:
            for lit in clause:
                if abs(lit) not in assignment:
                    unassigned_vars.add(abs(lit))
        if not unassigned_vars:
            return float('inf')

        var = unassigned_vars.pop()
        new_clauses = [clause for clause in clauses if var not in [abs(lit) for lit in clause]]

        depth1 = backtrack(assignment.copy(), new_clauses)
        if depth1 != float('inf'):
            depth1 += 1

        depth2 = backtrack(assignment.copy(), new_clauses)
        if depth2 != float('inf'):
            depth2 += 1

        return min(depth1, depth2)

    return backtrack({}, F)

def run_trial(seed):
    random.seed(seed)
    n_values = [10, 12, 14, 16]
    ensemble_generators = [generate_random_3sat, generate_tseitin, generate_ordering_principle, generate_random_3sat]
    instances_tested = 0
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for generator in ensemble_generators:

```

```

    if generator == generate_random_3sat:
        m = int(5.0 * n)
        F = generator(n, m)
    else:
        F = generator(n)

    q_hat = compute_q_hat(F)
    norm_2 = compute_norm_2(q_hat)
    norm_4 = compute_norm_4(F, n)
    r_F = norm_4 / norm_2
    G_F = (3 ** (3/2) - r_F) / 3 ** (3/2)
    d_star_F = dpll_depth(F)

    if d_star_F == float('inf'):
        continue

    instances_tested += 1
    metric_value = math.log2(d_star_F + 1) / (n * G_F) if G_F > 0 else float('inf')
    metric_values.append(metric_value)

    if metric_value < 0.05:
        conjecture_holds = False
        counterexample = f"n={n}, ensemble={generator.__name__}, d*(F)={d_star_F}, G(F)={G_F}"

if not metric_values:
    return {
        "metric_name": "log2(d*(F)+1)/(n*G(F))",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

return {
    "metric_name": "log2(d*(F)+1)/(n*G(F))",
    "metric_value": sum(metric_values) / len(metric_values),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    trial_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        trial_results.append(result)

    metric_values = [r["metric_value"] for r in trial_results if r["instances_tested"] > 0]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_instances")
        sys.exit(0)

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))

```

```

support_fraction = sum(1 for r in trial_results if r["conjecture_holds"]) / len(trial_results)

if all(r["conjecture_holds"] for r in trial_results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
elif any(not r["conjecture_holds"] for r in trial_results):
    first_failing_seed = seeds[trial_results.index(next(r for r in trial_results if not r["con
    counterexample = next(r["counterexample"] for r in trial_results if not r["conjecture_holds
    print(f"RESULT: FALSEIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
else:
    print("RESULT: INCONCLUSIVE reason=unsupported")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df7c90d8.py", line 178, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df7c90d8.py", line 141, in run_trial
    norm_4 = compute_norm_4(F, n)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df7c90d8.py", line 81, in compute_norm_4
    product *= (1 + s * x[v - 1]) / 2
                  ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in compute_norm_4 before producing any data, so the pre-registered support condition across 480 instances could not be evaluated. | next: Fix the variable indexing bug in compute_norm_4 (likely a 1-based vs 0-based mismatch or variables exceeding n) and rerun the full 480-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	254204
2	preregistration	claude_max	opus	0	0	6604

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	claude_max	opus	0	0	5934
4	novelty	claude_max	opus	0	0	8924
5	test_gen	mistral	codestral-latest	0	0	316107
6	test_gen	mistral	codestral-latest	0	0	316271
7	test_gen	mistral	codestral-latest	0	0	313005
8	test_gen	mistral	codestral-latest	0	0	314630
9	judge	claude_max	opus	0	0	4276

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1539955 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in *research/audit/2b2653395609.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/2b2653395609.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/2b2653395609.tar.gz* (if generated)